

CS 247 Assignment 5 Report

Abdullah Hejazi

The homework is about visualizing 2D vector fields and scalar fields by taking both steady (static) data and unsteady (time-dependent) data and visually represent how particles and flows move through these fields using glyphs, streamlines and pathlines.

I broke down the implementation to three main concepts that we covered in class:

- **Data Access and Interpolation:** Because the datasets provide vectors only at whole-number grid points, I needed a way to calculate the vector forces acting on a particle at continuous decimal coordinates. For steady flow I implemented bilinear interpolation to take a weighted average of the 4 closest neighbors. For unsteady flow I implemented trilinear interpolation to average 8 neighbors across space and time.
- **Numerical Integration:** This has to do with the Ordinary Differential Equation (ODE) the particle's velocity. I implemented two integration methods to step the particle forward which are the Euler method and the Runge-Kutta 2 (RK2) method.
- **Visual Mapping and Colormaps:** I also implemented the transfer functions in the fragment shaders normalized scalar values (0.0 to 1.0) to RGB colors. I implemented arrow glyphs for the vectors with changing lengths depending on vector magnitude, using sub-sampling.

Output

The visual outputs represent different properties of the flow:

- **Streamlines:** Time is frozen here so the output shows the trajectory a particle would take if the currents in the currently loaded time-slice never changed.
- **Pathlines:** Here, time is changing. The output shows the actual path a particle takes as the underlying vector field shifts and evolves like the hurricane moving in the grid over time.
- **Glyph:** The background color is the intensity of the scalar field at the location. The arrow glyphs show the instantaneous direction and speed of the flow at sampled intervals.

Implementation Challenges and Solutions

I faced difficulties while coding the integration and line-tracing logic. I overcame them by debugging and reviewing the lecture slides:

- My pathlines looked exactly the same as my streamlines. I solved this by changing the computePathline function to track a specific particle time variable that increments by dt with every step. This allowed the trilinear interpolation to transition between the different data time-slices
- The program would freeze when drawing streamlines. I realized the particles were either flying off the edge of the grid or getting stuck where the vector magnitude was zero I fixed this by stopping conditions

Sample Outputs





